



南開大學
Nankai University

高级语言程序设计 实验报告

姓名：张露桐

学号：2511612

学院：密码与网络空间安全学院

专业：密码科学与技术

2026 年 5 月 13 日

目 录

1 题目	2
2 开发软件	2
3 课题要求	2
4 设计思想	2
4.1 系统整体框架	2
4.2 各主要模块功能及调用关系说明	4
4.3 游戏运行流程	4
4.3.1 游戏整体流程	4
4.3.2 解密流程	4
4.3.3 AI 对话流程	5
4.3.4 可视化流程	5
4.4 信号-槽机制设计	5
4.4.1 核心信号连接关系	5
5 游戏运行	7
5.1 游戏开始	7
5.2 第一关与第二关地图	7
5.3 凯撒密码可视化	7
5.4 栅栏密码可视化	8
5.5 维吉尼亚密码可视化	8
5.6 Mini-AES 密码可视化	9
5.6.1 加密	9
5.6.2 解密	9
5.7 AI 对话系统	10
6 收获	10
7 AI 使用情况	11
7.1 AI 工具名称	11
7.2 使用 AI 工具完成内容	11

1 题目

基于 Qt 框架的密码学主题闯关游戏设计与实现

2 开发软件

- 开发语言：C++ (标准 C++17 及以上)
- 开发框架：Qt 6.10.1 (MinGW 64-bit)
- IDE：Qt Creator
- 图形引擎：QGraphicsView Framework (用于处理 2D 游戏场景、精灵动画及物理碰撞)
- 版本控制：Git (采用 dev 分支管理模式)
- 外部接口：DeepSeek LLM API

3 课题要求

本课题旨在结合计算机专业知识、密码学与软件工程实践，开发一款兼具教育性与趣味性的游戏。具体要求包括：

1. 多关卡设计：实现关卡的循序渐进，每一关对应一种经典的密码学算法。
2. 核心算法实现：在游戏逻辑中集成凯撒密码 (Caesar Cipher)、栅栏密码 (Rail Fence Cipher)、维吉尼亚密码 (Vigenère cipher)、对称加密算法 (Symmetric-key Cryptography) 等解密环节。
3. 游戏性要素：包含 2D 角色控制、障碍物避让、关卡解锁机制及视频过场动画。
4. 智能辅助系统：接入大语言模型 (LLM)，根据玩家当前的尝试次数和游戏状态，动态提供分级提示 (Hint System)。

4 设计思想

4.1 系统整体框架

```
1 main.cpp (程序入口)
2 |
3 | └─ StartScreen (开始菜单)
4 |   |
5 |   └─ [点击"开始游戏"] --┐
6 |                           ▼
7 |           MainWindow (主游戏窗口 / 总指挥)
8 |               |
9 |               |-- (核心驱动) QTimer --(timeout)--> advance()
10 |               |
11 |               |-- (初始化/管理) -----┐
12 |               | |-- LevelManager (关卡与对象管理)
```

```

13 | | |-- [创建] -- Player (玩家角色)
14 | | | |-- deadHandler (死亡通知)
15 | | | |-- reachedDoorHandler (通关)
16 | | | └─ arrowRequestHandler (触发)
17 | | |-- [创建] -- TerrainBlock (地形/门)
18 | | └─ [创建] -- Arrow (箭矢陷阱)
19 | |
20 | |-- InputController (输入控制器)
21 | | └─ [接收按键] --> 控制 Player 移动/跳跃
22 | |
23 | |-- PuzzleManager (谜题调度中心)
24 | | |-- [调度] -- CaesarPuzzle (凯撒谜题)
25 | | | |-- [点击提示] --> AIChatDialog
26 | | | └─ [检查答案] --> UIManager
27 | | └─ [调度] -- RailFencePuzzle (栅栏谜题)
28 | | | |-- [点击提示] --> AIChatDialog
29 | | | └─ [检查答案] --> UIManager
30 | | └─ [调度] -- VigenerePuzzle (维吉尼亚谜题)
31 | | | |-- [点击提示] --> AIChatDialog
32 | | | └─ [检查答案] --> UIManager
33 | | └─ [调度] -- MiniAESPuzzle (简化对称加密谜题)
34 | | | |-- [点击提示] --> AIChatDialog
35 | | | └─ [检查答案] --> UIManager
36 | |
37 | |-- UIManager (通用 UI 弹窗管理)
38 | | └─ [弹出] -- DeathDialog (死亡/重试)
39 | |
40 | |-- VideoController (过场动画控制)
41 | |
42 | └─ AIManager (AI 网络请求核心)
43 | | |-- [接收请求] --> AIChat / Puzzle
44 | | └─ [发射信号] hintReady --> GameWindow
45 |
46 └─ VideoController (过场动画控制器)

```



```
2      ↓
3  Puzzle 接收输入
4      ↓
5  调用算法模块（加密/解密）
6      ↓
7  生成结果
8      ↓
9  与标准答案比对
10     ↓
11  发出结果信号（正确/错误）
```

4.3.3 AI 对话流程

```
1  用户点击“AI提示”
2      ↓
3  UIManager 捕获事件
4      ↓
5  调用 AIManager::requestHint()
6      ↓
7  AIManager 生成提示内容
8      ↓
9  返回 UI 显示
```

4.3.4 可视化流程

```
1  用户触发“显示过程”
2      ↓
3  Puzzle 调用可视化模块
4      ↓
5  生成中间状态数据
6      ↓
7  UI逐步渲染（动画/网格）
8      ↓
9  用户观察算法执行过程
```

4.4 信号-槽机制设计

这是本系统的重要设计部分，通过 Qt 的信号-槽机制实现模块间解耦通信。

4.4.1 核心信号连接关系

1. 开始游戏

```
1  StartButton.clicked
2      ↓
3  GameWindow::startGame()
4      ↓
5  LevelManager::loadLevel()
```

2. 关卡切换

```
1 LevelManager::levelChanged(int)
2     ↓
3 PuzzleManager::loadPuzzle(int)
4     ↓
5 UIManager::updateUI()
```

3. 提交答案

```
1 SubmitButton.clicked
2     ↓
3 Puzzle::checkAnswer(QString)
4     ↓
5 emit answerResult(bool)
```

4. 答案反馈

```
1 Puzzle::answerResult(bool)
2     ↓
3 GameWindow::handleResult(bool)
4     ↓
5 (true) → LevelManager::nextLevel()
6 (false) → UI 提示 错误
```

5. AI 提示

```
1 HintButton.clicked
2     ↓
3 AIManager::generateHint()
4     ↓
5 emit hintReady(QString)
6     ↓
7 UIManager::showHint()
```

6. 可视化更新

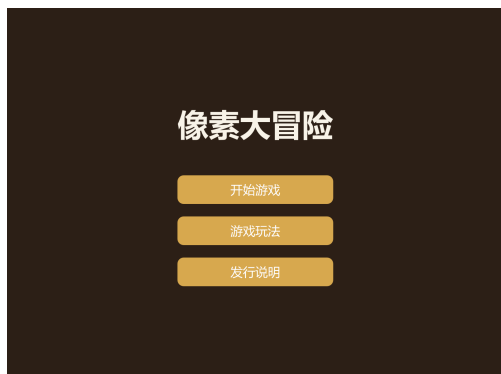
```
1 Puzzle::visualStepReady(data)
2     ↓
3 UIManager::updateVisualization(data)
```

7. 视频播放

```
1 GameWindow::startVideo()
2     ↓
3 VideoController::play()
4     ↓
5 emit videoFinished()
6     ↓
7 GameWindow::continueGame()
```

5 游戏运行

5.1 游戏开始

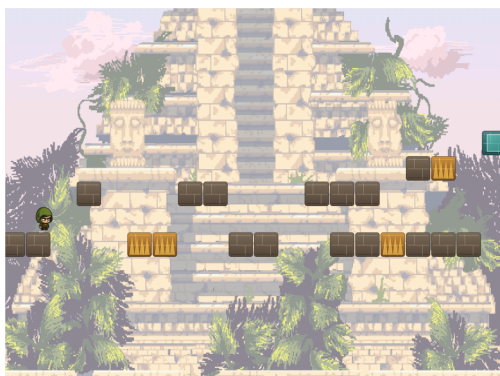


(a) 游戏主界面

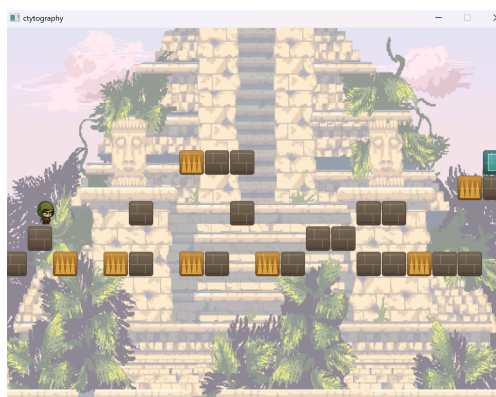


(b) 关卡选择与进度重置

5.2 第一关与第二关地图



(a) 第一关



(b) 第二关

5.3 凯撒密码可视化



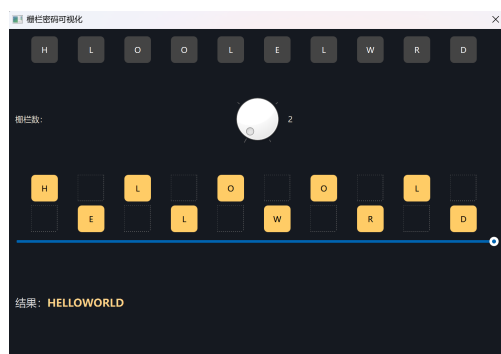
(a) 偏移前



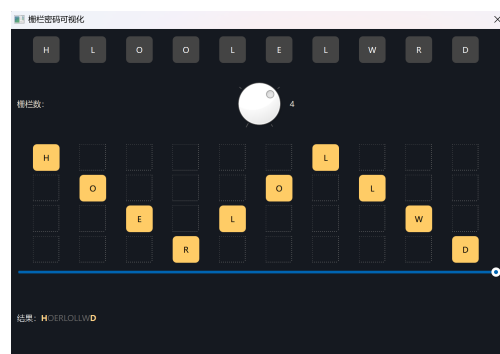
(b) 偏移后

转动转盘，偏移量调节到正确位置，字母变为绿色，转盘变亮

5.4 栅栏密码可视化



(a) 栅栏数为 2



(b) 栅栏数为 4

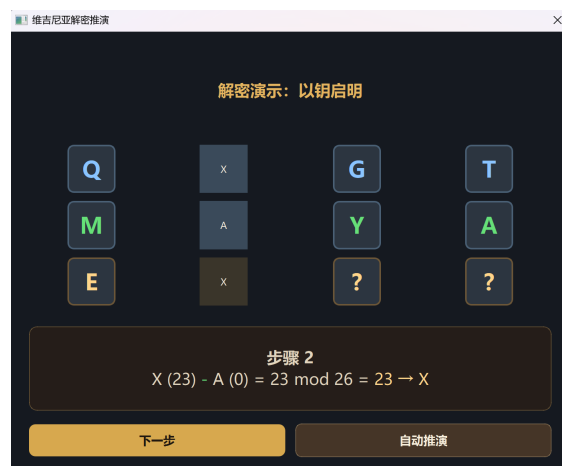
栅栏数为 2 时，所有字母填入正确位置，单词显示金色高亮；栅栏数为 4 时，部分字母放置错误，显示灰色

```
1 for (int i = 0; i < result.size(); i++) {
2     // 正确字符亮色高亮，错误字符灰色显示
3     if (i < targetText.size() && result[i] == targetText[i])
4         richText += "<span style='color:#ffd58a;'>" + QString(result[i]) + "</span>";
5     else
6         richText += "<span style='color:#666;'>" + QString(result[i]) + "</span>";
7 }
```

5.5 维吉尼亚密码可视化



(a) 前进相应偏移量



(b) 后退相应偏移量

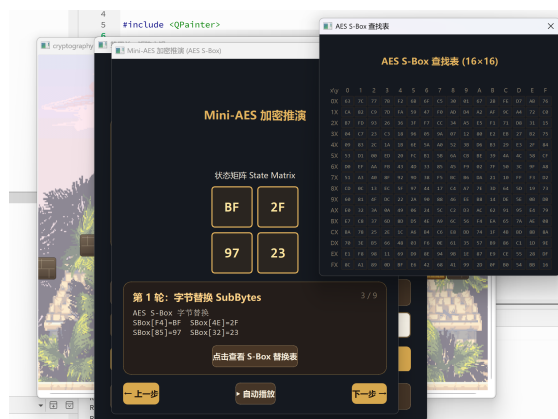
加密时按照密钥前进一定偏移量；解密时按照密钥后退一定偏移量

5.6 Mini-AES 密码可视化

5.6.1 加密



(a) 加密关卡 ui 设计



(b) 加密步骤推演

加密环节可视化展示了以下算法步骤：

- 第 0 轮 $\rightarrow$ AddRoundKey(RK0)
- 第 1 轮 $\rightarrow$ SubBytes(S-Box) $\rightarrow$ ShiftRows(行 1 两元素互换) $\rightarrow$ MixColumns(同列两元素互换后 Xor) $\rightarrow$ AddRoundKey(RK1)
- 第 2 轮 $\rightarrow$ SubBytes $\rightarrow$ ShiftRows $\rightarrow$ AddRoundKey(RK2) $\rightarrow$ 输出密文

5.6.2 解密



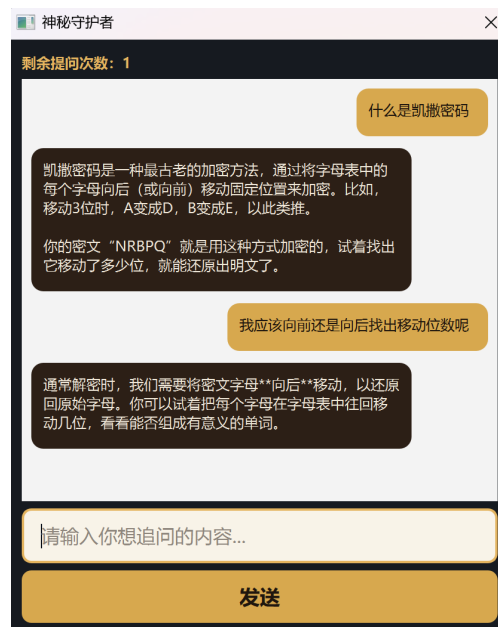
(a) 解密关卡 ui 设计



(b) 解密步骤推演

解密步骤是加密步骤的逆序

5.7 AI 对话系统



设置三次 AI 对话限制，设置提示词，对玩家进行进阶式回答

6 收获

1. 加深了对 C++ 面向对象编程的理解

- 继承：例如 `class GameWindow : public QGraphicsView`
- 多态与重写：Qt 的图形框架会在每一帧自动调用某些虚函数，通过重写它们来注入自己的逻辑
- 封装：降低了模块之间的耦合

2. 学习了主要的 QT 库的使用

(a) QT 核心模块：

- QObject：为所有 QT 类的对象提供信号槽机制
- QString：字符串处理
- QTimer：定时器，用于游戏循环、动画延时

(b) Qt GUI 模块

- QKeyEvent：键盘输入处理
- QPixmap：加载和显示图片资源

(c) Qt Widgets 模块

- QGraphicsScene：游戏场景，管理所有游戏对象
- QDialog：制作对话框

(d) Qt Network 模块：实现与网络的通信，用于 HTTP 请求

3. 学习了信号槽机制、lambda 表达式等的使用

```
1 //Player.cpp——信号源
2 if (type == "Arrow" && hitBox.intersects(itemRect)) {
3     isDead = true;
4     if (deadHandler) deadHandler();
5     return;
6 }
7
8 //Levelmanager.cpp——信号传递
9 player->setDeadHandler([this]() {
10     emit playerDead();
11 });
12
13 //Gamewindow.cpp——信号接收
14 connect(levelManager, &LevelManager::playerDead, this, &GameWindow::
    onPlayerDead);
```

进行了解耦设计，分开了玩家死亡时的信号传递。

7 AI 使用情况

7.1 AI 工具名称

Google gemini

7.2 使用 AI 工具完成内容

1. 使用 AI 工具逐行拆解了一个学长的 QT 项目，明白了简单的 QT 库的用法，以及一些 QT 不同于课内 C++ 的语法，例如信号槽机制、lambda 表达式等
2. 对于每一个要编写的关卡部分，我会询问 AI 生成一个关于实现基本功能的样例。例如，对于“如何制作 startscreen 部分”，我会进行如下询问：

生成一个简单但是全面的例子来教会我如何使用 QT 编写 startscreen 的界面，C++ 语言。包括“开始游戏”按钮，连接游戏界面；“游戏玩法”按钮，点击之后有弹窗；按钮布局设置；界面颜色设置；游戏大标题设置

我将 AI 工具当成一个为我的项目量身定做的 QT 使用书籍，通过有针对性地提供样例来帮助我实现项目，而不是替我编写

3. 利用 AI 工具修改程序错误或游戏 bug。例如，当我使用错误的库来导入背景图片，游戏界面变得异常卡顿，AI 建议我更换了图形渲染库，并调整了背景图片的加载方式，最终背景成功设置。